Error Handling

[??]

Overview

Identifying and handling run-time errors is important for developing robust applications. There can be multiple kinds of run-time errors:

- Recoverable errors:
  - Errors indicated by functions through return values (error codes)
  - C++ exceptions, thrown using `throw` keyword
- Unrecoverable (fatal) errors:
  - Failed assertions (using `assert` macro and equivalent methods, see Assertions) and `abort()` calls.
  - CPU exceptions: access to protected regions of memory, illegal instruction, etc.
  - System level checks: watchdog timeout, cache access error, stack overflow, stack smashing, heap corruption, etc.

This guide primarily introduces the error handling mechanisms in ESP-IDF for **recoverable errors** and provides common error handling patterns. Some sections also mention macros used for **unrecoverable errors**, with the aim of illustrating their use in scenarios with different levels of error severity.

For instructions on diagnosing unrecoverable errors, see Fatal Errors.

Error Codes

The majority of ESP-IDF-specific functions use `esp_err_t` type to return error codes. `esp_err_t` is a signed integer type. Success (no error) is indicated with `ESP_OK` code, which is defined as zero.

Various ESP-IDF header files define possible error codes using preprocessor defines. Usually these defines start with `ESP_ERR_` prefix. Common error codes for generic failures (out of memory, timeout, invalid argument, etc.) are defined in `esp_err.h` file. Various components in ESP-IDF may define additional error codes for specific situations.

For the complete list of error codes, see Error Code Reference.

Converting Error Codes to Error Messages

For each error code defined in ESP-IDF components, `esp_err_t` value can be converted to an error code name using `esp_err_to_name()` or `esp_err_to_name_r()` functions. For example, passing `0x101` to `esp_err_to_name()` will return a `ESP_ERR_NO_MEM` string. Such strings can be used in log output to make it easier to understand which error has happened.

Additionally, `esp_err_to_name_r()` function will attempt to interpret the error code as a standard POSIX error code, if no matching `ESP_ERR_` value is found. This is done using `strerror_r` function. POSIX error codes (such as `ENOENT`, `ENOMEM`) are defined in `errno.h` and are typically obtained from `errno` variable. In ESP-IDF this variable is thread-local: multiple FreeRTOS tasks have their own copies of `errno`. Functions which set `errno` only modify its value for the task they run in.

This feature is enabled by default, but can be disabled to reduce application binary size. See CONFIG_ESP_ERR_TO_NAME_LOOKUP. When this feature is disabled, `esp_err_to_name()` and `esp_err_to_name_r()` are still defined and can be called. In this case, `esp_err_to_name()` will return `UNKNOWN ERROR`, and `esp_err_to_name_r()` will return `Unknown error 0xXXXX(YYYYY)`, where `0xXXXX` and `YYYYY` are the hexadecimal and decimal representations of the error code, respectively.

Macro For Unrecoverable Errors

The `ESP_ERROR_CHECK` macro, defined in `esp_err.h`, is used to handle unrecoverable errors in ESP-IDF applications. It functions similarly to the standard `assert` macro, but specifically checks whether an `esp_err_t` value is equal to `ESP_OK`. If the value is not `ESP_OK`, `ESP_ERROR_CHECK` prints a detailed error message and calls `abort()`, terminating the application.

The behavior of `ESP_ERROR_CHECK` can be controlled using assertion-related configuration options:

- If `CONFIG_COMPILER_OPTIMIZATION_ASSERTIONS_ENABLE` is set (default), the macro prints an error message and terminates the program.
- If `CONFIG_COMPILER_OPTIMIZATION_ASSERTIONS_SILENT` is enabled, the program terminates silently without printing an error message.
- If `CONFIG_COMPILER_OPTIMIZATION_ASSERTIONS_DISABLE` (`NDEBUG` is defined), the macro only prints an error message and does not terminate the program.

Use `ESP_ERROR_CHECK` in situations where an error is considered fatal and the application cannot continue safely. For situations where the application can recover from an error, use the macros described in the next section.

`ESP_ERROR_CHECK_WITHOUT_ABORT`

The `ESP_ERROR_CHECK_WITHOUT_ABORT` macro, defined in `esp_err.h`, is closely related to the **Macros For Recoverable Errors**. The macro behaves similarly to `ESP_ERROR_CHECK`, but instead of terminating the program with `abort()`, it prints an error message in the same format and returns the error code if the value is not `ESP_OK`. This allows the application to continue running, making it suitable for cases where errors should be reported but are not considered fatal.

The behavior of `ESP_ERROR_CHECK_WITHOUT_ABORT` is controlled by the same assertion-related configuration options as `ESP_ERROR_CHECK`. If either `CONFIG_COMPILER_OPTIMIZATION_ASSERTIONS_DISABLE` or `CONFIG_COMPILER_OPTIMIZATION_ASSERTIONS_SILENT` is enabled, the macro does not print any error message, otherwise, the macro prints an error message.

Use `ESP_ERROR_CHECK_WITHOUT_ABORT` when you want to print errors for diagnostic purposes without stopping the application.

Error message will typically look like this:

```
ESP_ERROR_CHECK failed: esp_err_t 0x107 (ESP_ERR_TIMEOUT) at 0x400d1fdf

file: "/Users/user/esp/example/main/main.c" line 20
func: app_main
expression: sdmmc_card_init(host, &card)

Backtrace: 0x40086e7c:0x3ffb4ff0 0x40087328:0x3ffb5010 0x400d1fdf:0x3ffb5030
0x400d0816:0x3ffb5050
```

Note

If ESP-IDF monitor is used, addresses in the backtrace will be converted to file names and line numbers.

- The first line mentions the error code as a hexadecimal value, and the identifier used for this error in source code. The latter depends on CONFIG_ESP_ERR_TO_NAME_LOOKUP option being set. Address in the program where error has occurred is printed as well.
- Subsequent lines show the location in the program where `ESP_ERROR_CHECK` macro was called, and the expression which was passed to the macro as an argument.
- Finally, backtrace is printed. This is part of panic handler output common to all fatal errors. See Fatal Errors for more information about the backtrace.

Macros For Recoverable Errors

For recoverable errors, ESP-IDF provides a set of macros defined in `esp_check.h`. The **ESP_RETURN_ON_...**, **ESP_GOTO_ON_...**, **ESP_RETURN_VOID_ON_...**, and **ESP_RETURN_ON_ERROR_CLEANUP** macros enable concise and consistent error handling, improving code readability and maintainability. Unlike `ESP_ERROR_CHECK`, these macros do not terminate the program; instead, they print an error message and return or jump as appropriate. For use in interrupt service routines (ISRs), corresponding `_ISR` versions (such as `ESP_RETURN_ON_ERROR_ISR`) are available, ensuring safe operation in interrupt contexts.

The macros are defined as follows:

- **ESP_RETURN_ON_...**: Return from the function if an error or failed condition is detected:
    - `ESP_RETURN_ON_ERROR` - Checks an error code; if not `ESP_OK`, prints a message and returns the error code.
    - `ESP_RETURN_ON_FALSE` - Checks a condition; if false, prints a message and returns the supplied `err_code`.
    - `ESP_RETURN_ON_ERROR_ISR` - For ISR context.
    - `ESP_RETURN_ON_FALSE_ISR` - For ISR context.
- **ESP_GOTO_ON_...**: Jump to a label if an error or failed condition is detected:
    - `ESP_GOTO_ON_ERROR` - Checks an error code; if not `ESP_OK`, prints a message, sets `ret` to the code, and jumps to `goto_tag`.
    - `ESP_GOTO_ON_FALSE` - Checks a condition; if false, prints a message, sets `ret` to `err_code`, and jumps to `goto_tag`.
    - `ESP_GOTO_ON_ERROR_ISR` - For ISR context.
    - `ESP_GOTO_ON_FALSE_ISR` - For ISR context.
- **ESP_RETURN_VOID_...**: Return from a `void` function if an error or failed condition is detected:
    - `ESP_RETURN_VOID_ON_ERROR` - Checks an error code; if not `ESP_OK`, prints a message and returns.
    - `ESP_RETURN_VOID_ON_FALSE` - Checks a condition; if false, prints a message and returns.
    - `ESP_RETURN_VOID_ON_ERROR_ISR` - For ISR context.
    - `ESP_RETURN_VOID_ON_FALSE_ISR` - For ISR context.
- **ESP_RETURN_ON_ERROR_CLEANUP**: Return from the function after executing cleanup code if an error is detected. This macro is useful when resources need to be released before returning an error code. It evaluates an expression that returns `esp_err_t`, and if the result is not `ESP_OK`, it executes the cleanup code provided in the variable arguments (such as freeing resources or logging) before returning the error code from the current function.

The default behavior of these macros can be adjusted: if the CONFIG_COMPILER_OPTIMIZATION_CHECKS_SILENT option is enabled in Kconfig, error messages will not be included in the application binary and will not be printed.

Error Handling Examples

Some examples

```
static const char* TAG = "Test";

esp_err_t test_func(void)
{
    esp_err_t ret = ESP_OK;

    ESP_ERROR_CHECK(x);                                         // err message printed
if `x` is not `ESP_OK`, and then `abort()`.
    ESP_ERROR_CHECK_WITHOUT_ABORT(x);                           // err message printed
if `x` is not `ESP_OK`, without `abort()`.
    ESP_RETURN_ON_ERROR(x, TAG, "fail reason 1");               // err message printed
if `x` is not `ESP_OK`, and then function returns with code `x`.
    ESP_GOTO_ON_ERROR(x, err, TAG, "fail reason 2");            // err message printed
if `x` is not `ESP_OK`, `ret` is set to `x`, and then jumps to `err`.
    ESP_RETURN_ON_FALSE(a, err_code, TAG, "fail reason 3");     // err message printed
if `a` is not `true`, and then function returns with code `err_code`.
    ESP_GOTO_ON_FALSE(a, err_code, err, TAG, "fail reason 4");  // err message printed
if `a` is not `true`, `ret` is set to `err_code`, and then jumps to `err`.

    ESP_RETURN_ON_ERROR_CLEANUP(init_resource(), free_mem(), deinit_resource()); // if
failure, clean up resources by calling `free_mem()` and `deinit_resource()`, then
return the error code.

    ESP_RETURN_ON_ERROR_CLEANUP(                                // if more complex
cleanup is needed, use a `do {...} while(0)` block, (required for ESP_LOG usage).
        sensor_calibrate(),
        do {
            if (err_rc_ == ESP_ERR_TIMEOUT) {
                ESP_LOGE(TAG, "Sensor calibration timeout");
            }
            sensor_deinit();
            ESP_LOGE(TAG, "Sensor cleanup completed");
        } while (0)
    );

err:
    // clean up
    return ret;
}
```

Error Handling Patterns

1. Attempt to recover. Depending on the situation, we may try the following methods:

- retry the call after some time;
- attempt to de-initialize the driver and re-initialize it again;
- fix the error condition using an out-of-band mechanism (e.g reset an external peripheral which is not responding).

Example:

```
esp_err_t err;
do {
    err = sdio_slave_send_queue(addr, len, arg, timeout);
    // keep retrying while the sending queue is full
} while (err == ESP_ERR_TIMEOUT);
if (err != ESP_OK) {
    // handle other errors
}
```

2. Propagate the error to the caller. In some middleware components this means that a function must exit with the same error code, making sure any resource allocations are rolled back.

   Example:

```
sdmmc_card_t* card = calloc(1, sizeof(sdmmc_card_t));
if (card == NULL) {
    return ESP_ERR_NO_MEM;
}
esp_err_t err = sdmmc_card_init(host, &card);
if (err != ESP_OK) {
    // Clean up
    free(card);
    // Propagate the error to the upper layer (e.g., to notify the user).
    // Alternatively, application can define and return custom error code.
    return err;
}
```

3. Convert into unrecoverable error, for example using `ESP_ERROR_CHECK`. See Macro For Unrecoverable Errors section for details.

Terminating the application in case of an error is usually undesirable behavior for middleware components, but is sometimes acceptable at application level.

Many ESP-IDF examples use `ESP_ERROR_CHECK` to handle errors from various APIs. This is not the best practice for applications, and is done to make example code more concise.

Example:

```
ESP_ERROR_CHECK(spi_bus_initialize(host, bus_config, dma_chan));
```

C++ Exceptions

See Exception Handling.

API Reference

See API Reference.

Was this page helpful?